

Production-Ready Agents

Prompt (Original Article)

There is a persistent gap between the AI agents that perform well in a demonstration and the ones that hold up under real production traffic.

This is because the systems handling live customer workloads tend to depend on the language model far less than their presentation suggests. Most of their behavior runs through conventional, deterministic code, with the model invoked at a small number of specific decision points. The difference between a dependable agent and a fragile one often comes down to a set of engineering decisions about where the model is given control and where it is held back.

But how do you build agents that work reliably in production? Are there any best practices?

In 2011, Heroku's co-founder published the Twelve-Factor App, a set of principles for building web services that deploy and scale reliably, and that document influenced how a generation of engineers thought about software.

A parallel effort later appeared for AI in the form of a widely shared set of principles called the Twelve-Factor Agents. These principles were aimed at making language-model applications robust enough for production. The specifics have kept evolving since, and teams at companies like Anthropic, Cognition, and Intercom have added lessons of their own.

In this article, we try to explore the collective thinking into a smaller set of practices and explain the reasoning behind each one, rather than asking anyone to memorize a numbered list. The practices fall into four areas, with a final section on the tradeoffs. Here's what we will cover:

- How does context control what the model sees on every call?
- How does the control flow keep the loop and its stopping conditions in deterministic code?
- How does the state hold memory in software while the model stays stateless?
- How does the scope keep each agent narrow and supervised?

By the end, the article will have built a working definition of a production agent and a clear view of which decisions carry the most weight.

Disclaimer: This post is based on publicly shared details from various sources. References at the end. Please comment if you notice any inaccuracies.

The Loop

Let us start with the simplest agent anyone can build, because it reveals the structure that everything else refines.

At its core, an agent is a loop. The model receives some context, which is the running record of what has happened so far, and it returns a single decision about the next step to take. That decision arrives as structured output. It is machine-readable data, such as JSON rather than free-flowing prose, so that ordinary code can read it and act on it. The surrounding code executes the requested step, appends the

result to the context, and returns everything to the model for the next decision. The loop continues until the model signals that the work is finished.

A useful way to picture the model's role is as a function that starts fresh on every call.

On each call, the model sees only the context provided to it, makes one decision, and discards everything the moment it responds. The context window, which is the bundle of text the model reads on a given call, serves as the entire memory of the system.

The appeal of this design is clear. A developer describes a goal and supplies a set of tools, and the model works out the order of operations on its own. The design requires less branching logic, since the model handles the decisions. This is the version of an agent that can perform quite well in a demo setup. However, the trouble begins when real traffic arrives.

Failure Modes

The design we looked at in the previous section fails under production conditions, and the failures follow some predictable patterns:

- Compounding error multiplies across chained steps.
- Confidently wrong output reaches a user.
- A loop runs longer than intended.
- State disappears when the plan lives only inside the model.

Let's understand them in a little more detail.

The first pattern is compounding error. Every model call carries some chance of a wrong decision, and a freewheeling agent chains many calls together. Suppose each step is correct 95 percent of the time, which sounds reliable. Run twenty such steps in sequence, and the odds that all of them succeed fall to roughly one in three. The math is multiplicative, so reliability that looks fine in isolation degrades quickly across a long chain. This fact explains why production teams add so many guardrails.

The second pattern is confidently wrong output reaching a user. In 2025, the support agent for the coding tool Cursor told customers about a one-device-per-subscription rule that the company later confirmed it had fabricated, and a wave of cancellations followed. A year earlier, a tribunal held Air Canada liable after its chatbot invented a bereavement-fare policy, and ordered the airline to honor it. Studies place hallucination rates, meaning the rate at which a model produces plausible but false statements, somewhere between 3 and 27 percent, even in controlled settings. An agent that speaks directly to customers turns that rate into a big business risk.

The final two patterns are harder to notice, but just as costly.

A loop with weak stopping conditions can run far longer than intended, consuming tokens and money with each pass. And lastly, when the plan lives only inside the model's context, a crash or a long-running task can lose the entire thread of work.

Each failure points to a specific practice, grouped into the four areas below, beginning with the inputs.

Context

The context window is everything the model knows on a given call, so controlling its contents is the first and largest lever on reliability. Three habits do most of the work here.

The first is to own your prompts. A prompt is the instruction text sent to the model, and it deserves the same treatment as any other source code, which means version control, review, and testing. Many agent frameworks generate prompts automatically and keep them out of sight, which is convenient until behavior changes and the cause is hard to locate. Keeping prompts in the application's own codebase keeps their effects reproducible.

The second habit is to own the context window itself. The contents of each call are a deliberate choice, and that choice carries more weight than it first appears. A model given a focused, relevant context performs better than the same model handed a large pile of loosely related history. Quality degrades as the window fills with marginal material. Therefore, deliberate pruning by actively removing content beyond what the current step requires preserves the model's accuracy. In other words, relevance beats volume on almost every call.

The third habit concerns tools. A tool is simply a function the model can request, described with a clear name and a schema for its inputs. When those descriptions are precise, the model selects the right tool and fills its arguments correctly. When they are vague, the model is more likely to choose the wrong one. Treating tool definitions as a careful piece of interface design pays off immediately.

A discipline called context engineering has grown up around these habits, and it's a big topic in itself. However, the core idea stays simple: decide what the model sees, on purpose, every time.

Control Flow

Owning the inputs matters only if the surrounding code decides when the model runs and when the loop ends. In a dependable agent, the control flow belongs to the deterministic code around the model, and the model is consulted at a few chosen moments.

Picture the overall flow as ordinary code, a sequence of steps, conditionals, and calls to external systems. At two or three points in that flow, where a decision genuinely calls for judgment, the system invokes the model. Everywhere else, plain deterministic logic does the work because it is cheaper to run, predictable in its output, and straightforward to test.

Two practical rules make it work:

- First, every loop needs an escape hatch, which is a hard limit that forces it to stop. A cap on the number of iterations, a timeout, and explicit completion conditions together guarantee the agent halts even when the model would continue indefinitely.
- Second, invoke the model on purpose. The model belongs where the problem requires open-ended reasoning, and ordinary code handles everything that can be specified in advance.

Intercom's customer service product shows the pattern at scale.

Its newer Procedures let the agent reason in natural language while the surrounding system supplies deterministic controls, including conditional steps for decision points, small code snippets that guarantee the same input always yields the same output, and checkpoints where the agent pauses for human approval before a sensitive action.

To summarize, it is beneficial to start with the simplest version that solves the problem, and add model-driven autonomy only where simpler logic falls short.

State

Once the surrounding code owns the loop, a natural question follows. Where does the agent's memory live?

The answer is to keep the model stateless and to hold the state in software that the application controls.

Recall that the model starts each call fresh and reads only the context provided to it. That property, when treated as a feature, becomes powerful. The application stores the real state of the work, the conversation so far, the plan, the progress, and reconstructs the context from it on every call. Since the state lives in serializable storage (a form the system can save and reload), an agent can pause partway through a task and resume later, or recover cleanly after a crash, by loading the saved state and continuing.

This habit also keeps two kinds of states aligned. There is the model's view of what is happening, and there is the application's record of ground truth, such as the actual contents of a database. Keeping these two in agreement prevents the model from acting on a stale or inaccurate view of the current state.

There is a scaling benefit as well.

When the state lives entirely outside the model, any running instance of the service can pick up any request, because everything needed is available with the context. That allows many copies to run behind a load balancer, handling far more traffic.

A clean way to think about the model here is as a function that takes the current state and a new event and returns the next state along with an action to perform. The same inputs produce the same result, which makes the whole system easier to test and to trust.

Scope

Owning the inputs, the loop, and the state still leaves one decision open. How much should a single agent try to accomplish? The good answer favors small, focused agents kept under supervision over one broad agent asked to do everything.

A narrow agent with a single, well-defined job has a smaller surface for failure. It is easier to test, easier to reason about, and easier to fix when it misbehaves. When a task spans several distinct jobs, several narrow agents compose under deterministic orchestration. The surrounding code decides which agent runs and when, rather than assigning one general agent a long list of responsibilities.

Klarna's customer service assistant illustrates the upside. It handled roughly 2.3 million conversations in its first month, and most of those followed predictable paths such as checking a refund status or tracking an order, which structured logic handles well, with the model reserved for the cases that truly need it.

Supervision is the other half of the scope.

A human handoff deserves to be designed as a first-class step. It is reached on purpose when an action is sensitive, when confidence is low, or when a customer asks for a person. Intercom builds this in directly, handing a conversation to a human automatically whenever continuing would be the riskier choice. Treating human involvement as a planned path, complete with the state needed to brief the person, turns it into a strength of the system rather than a sign that something went wrong.

Tradeoffs

Everything so far describes practices that work today. But there are also disagreements and a few open questions.

The main debate concerns single versus multiple agents.

In June 2025, the team behind the coding agent Devin argued against multi-agent designs because parallel sub-agents make independent decisions that then conflict with one another. One day later, Anthropic described its own multi-agent research system, which used roughly fifteen times the tokens of a single-agent approach and scored about 90 percent higher on certain research tasks.

The apparent contradiction resolved over the following months into a shared pattern. A single orchestrator owns the full context and spawns isolated, short-lived sub-agents, each of which completes one task and returns a summary. The lesson to understand is that small, focused agents work best under firm orchestration, while sub-agents that communicate directly with one another tend to produce conflicting results.

The deeper question comes from what Rich Sutton called the Bitter Lesson. It is the observation that general methods that rely on more computation tend to overtake handcrafted techniques over time. Applied here, it warns that some of the scaffolding built to compensate for today's models will become redundant as the models improve. That is partly true, and it is the strongest argument against over-engineering.

The counterpoint is that several of these problems persist regardless of model strength. A finite context window, the need to keep a figure on page five of a document consistent with the same figure on page eight hundred, the requirement to pause and resume safely, all of these survive better models.

Cost becomes another consideration for every decision. Autonomy and multi-agent designs spend more tokens, sometimes far more, so they earn their place only where the value of the task justifies the bill.

Conclusion

A production agent, reduced to its essentials, is mostly deterministic software that calls a language model at a few deliberate points. The design decisions lie in choosing those points and limiting how much the model decides on its own.

The path to that definition ran through the simplest possible agent and its predictable failures, with each failure answered by one practice:

- Control what the model sees.
- Own the loop and give it a hard stop.
- Hold the state in software while the model stays stateless.
- Keep each agent narrow and supervised.

Around these sit real tradeoffs, including the open argument over multiple agents and the steady improvement in models that will retire some of this work over time. A sound approach starts with the simplest design that solves the problem, measures where it falls short, and grants the model more autonomy only where that autonomy provides clear value.

References: - The Twelve-Factor App - 12-Factor Agents — HumanLayer - Anthropic — Building Effective Agents - Anthropic — How We Built Our Multi-Agent Research System - Cognition — Don't Build Multi-Agents - Intercom — What's New with Fin 3 - Rich Sutton — The Bitter Lesson - Cursor support-bot incident — eWeek - Air Canada chatbot ruling — AI Business

Response (Summary)

The core argument: There's a real gap between demo-worthy AI agents and ones that hold up in production. The reason is that reliable agents actually lean on the language model far less than people assume — most of the logic runs through conventional deterministic code, with the model called in at just a few key decision points.

Framing: The piece draws a parallel to Heroku's old Twelve-Factor App methodology, noting that a similar "Twelve-Factor Agents" framework emerged for AI, and companies like Anthropic, Cognition, and Intercom have contributed their own lessons since.

The basic agent loop: An agent is essentially a loop — the model gets context, returns a structured decision (like JSON), code executes it, and the result gets appended to context for the next round. The model itself is stateless; it only "remembers" what's in its context window on that call.

Why naive agents break in production, four failure modes: 1. **Compounding error** — even a 95%-reliable step, chained 20 times, drops overall success to roughly one in three. 2. **Confidently wrong answers reaching users** — cited real incidents like Cursor's support bot inventing a device-limit policy, and Air Canada being held liable for a chatbot's fabricated bereavement fare. 3. **Runaway loops** that burn tokens/money without proper stop conditions. 4. **Lost state** when the only record of progress lives inside the model's context and a crash or long task wipes it out.

The four practice areas that address these: - **Context** — treat prompts like source code (version-controlled, reviewed), keep the context window pruned to what's relevant, and write precise tool definitions so the model picks the right one. - **Control flow** — let deterministic code own the loop; call the model only at a few points needing real judgment; always build in hard stops (iteration caps, timeouts). - **State** — keep the model stateless and store real state (conversation, plan, progress) in the application itself, so work can pause, resume, or survive crashes, and so multiple service instances can share load. - **Scope** — favor several small, narrow agents over one broad one, and design human handoff as a deliberate, first-class path rather than a failure signal.

Named examples: Intercom's Procedures (deterministic checkpoints plus human approval gates), Klarna's assistant handling ~2.3 million conversations in its first month mostly via structured logic.

Tradeoffs discussed: The single-agent vs. multi-agent debate (Devin's team argued against multi-agent designs due to conflicting parallel decisions, while Anthropic's multi-agent research system used ~15x the tokens for a large performance gain — resolved via an orchestrator-plus-short-lived-subagents pattern). Also touches on the "Bitter Lesson" — that some current scaffolding may become unnecessary as models improve — balanced against the view that certain constraints (finite context, safe pause/resume, consistency) will persist regardless.

Conclusion: A production-ready agent is mostly ordinary deterministic software, with the model invoked sparingly and deliberately — start simple, then add autonomy only where it earns its cost.